

ASSESSMENT TOOL

INSURANCE CLAIM PRECEDENT SEARCH

FAISS AND ChromaDB SEMANTIC SEARCH BENCHMARK

Student Name	M. Someswar Reddy
Register Number	192472081

1. Aim

To benchmark FAISS and ChromaDB for retrieving similar insurance claim summaries and to study whether very different monetary amounts affect semantic similarity.

2. Objective

To build a dataset with more than 200 claim summaries, generate one common sentence-embedding matrix, index the same embeddings in FAISS and ChromaDB, run the same ten semantic queries, benchmark both databases, and analyze the monetary-amount special case.

3. Dataset

The project contains 240 synthetic insurance claim summaries. Each record includes claim ID, incident type, description, claim amount, location, vehicle type, severity and claim summary.

Source: Synthetic dataset generated for this assessment.

4. Embedding Model

sentence-transformers/all-MiniLM-L6-v2. The same model and the same embedding vectors are used for both databases.

5. Vector Databases

FAISS and ChromaDB are both indexed from the same embedding matrix so that the database comparison is based on the same semantic representation.

6. Design Parameters

Parameter	Value
Dataset size	240 records
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Top-k	5
Queries	10
FAISS index	IndexFlatIP on normalized embeddings
ChromaDB	Persistent collection using cosine distance
Special case	Same incident type with very different claim amounts

7. Architecture

Insurance Claim Dataset → Sentence Embeddings → Same Embeddings → FAISS / ChromaDB → Top-5 Retrieval → Benchmark → Comparison → Recommendation

8. Procedure

1. Load 240 claim records.

2. Create claim text for embedding.
3. Generate normalized embeddings with all-MiniLM-L6-v2.
4. Index the embeddings in FAISS.
5. Index the same embeddings in ChromaDB.
6. Run the same 10 queries on both databases.
7. Retrieve top 5 results.
8. Measure indexing time and average query latency.
9. Calculate Top-5 relevance using incident-type agreement.
10. Compare top-5 results for at least three queries.
11. Run the same-incident/different-amount special case.
12. Repeat with claim amounts excluded from embedding text.
13. Use the measured results for the final recommendation.

9. Python Program

```
import csv
import shutil
import time
from pathlib import Path

import faiss
import numpy as np
import chromadb
from sentence_transformers import SentenceTransformer

BASE_DIR = Path(__file__).resolve().parent
DATASET = BASE_DIR / "claims_dataset.csv"
CHROMA_DIR = BASE_DIR / "chroma_db"
OUTPUT_DIR = BASE_DIR / "outputs"
OUTPUT_DIR.mkdir(exist_ok=True)

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
TOP_K = 5

QUERIES = [
    "rear-end collision with damage to the rear bumper and trunk",
    "vehicle hit from the side at an intersection with door damage",
    "windshield cracked by road debris",
    "car damaged by heavy rain and flood water",
    "vehicle stolen from a parking area",
    "engine compartment fire caused extensive vehicle damage",
    "hailstorm caused dents on the roof and bonnet",
    "parked car vandalised with broken windows and scratches",
    "motorcycle collision causing front-end damage",
    "vehicle hit while parked in a parking area"
]

SPECIAL_QUERY = "rear-end collision causing rear bumper and trunk damage"

def load_dataset():
    with open(DATASET, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def build_text(row, include_amount=True):
    text = (
        f"Incident type: {row['Incident_Type']}. "
        f"Claim description: {row['Description']}. "
        f"Location: {row['Location']}. "
        f"Vehicle type: {row['Vehicle_Type']}. "
        f"Severity: {row['Severity']}."
    )
    if include_amount:
        text += f" Claim amount: INR {row['Claim_Amount_INR']}."
    return text
```

```

def relevance(top_rows):
    if not top_rows:
        return 0.0
    target = top_rows[0]["Incident_Type"]
    return sum(r["Incident_Type"] == target for r in top_rows) / len(top_rows)

def benchmark(include_amount=True):
    rows = load_dataset()
    texts = [build_text(r, include_amount) for r in rows]
    model = SentenceTransformer(MODEL_NAME)

    embeddings = model.encode(texts, normalize_embeddings=True, show_progress_bar=True)
    embeddings = np.asarray(embeddings, dtype="float32")

    t0 = time.perf_counter()
    faiss_index = faiss.IndexFlatIP(embeddings.shape[1])
    faiss_index.add(embeddings)
    faiss_index_time = time.perf_counter() - t0

    if CHROMA_DIR.exists():
        shutil.rmtree(CHROMA_DIR)
    client = chromadb.PersistentClient(path=str(CHROMA_DIR))
    collection = client.get_or_create_collection(name="insurance_claims", metadata={"hnsw:space": "cosine"})

    ids=[r["Claim_ID"] for r in rows]

    metadatas=[{"Incident_Type":r["Incident_Type"],"Claim_Amount_INR":int(r["Claim_Amount_INR"]), "Location":r["Location"], "Severity":r["Severity"]} for r in rows]

    t0=time.perf_counter()
    collection.add(ids=ids, embeddings=embeddings.tolist(), documents=texts, metadatas=metadatas)
    chroma_index_time=time.perf_counter()-t0

    faiss_results=[]; chroma_results=[]; fl=[]; cl=[]

    for query in QUERIES:
        q=np.asarray(model.encode([query],normalize_embeddings=True),dtype="float32")
        t0=time.perf_counter()
        scores,pos=faiss_index.search(q,TOP_K)
        fl.append(time.perf_counter()-t0)
        faiss_results.append([rows[int(p)] for p in pos[0]])

        t0=time.perf_counter()
        result=collection.query(query_embeddings=q.tolist(),n_results=TOP_K)
        cl.append(time.perf_counter()-t0)
        chroma_results.append([next(r for r in rows if r["Claim_ID"]==cid) for cid in result["ids"][0]])

    special_q=np.asarray(model.encode([SPECIAL_QUERY],normalize_embeddings=True),dtype="float32")
    _,pos=faiss_index.search(special_q,TOP_K)
    special_f=[rows[int(p)] for p in pos[0]]
    result=collection.query(query_embeddings=special_q.tolist(),n_results=TOP_K)
    special_c=[next(r for r in rows if r["Claim_ID"]==cid) for cid in result["ids"][0]]

    return {
        "rows":rows,
        "faiss_index_time":faiss_index_time,
        "chroma_index_time":chroma_index_time,
        "faiss_avg_latency":sum(fl)/len(fl),
        "chroma_avg_latency":sum(cl)/len(cl),
        "faiss_relevance":sum(relevance(x) for x in faiss_results)/len(faiss_results),
        "chroma_relevance":sum(relevance(x) for x in chroma_results)/len(chroma_results),
        "faiss_results":faiss_results,"chroma_results":chroma_results,
        "special_f":special_f,"special_c":special_c
    }

def save_outputs(with_amount, without_amount):
    with open(OUTPUT_DIR/"benchmark_results.csv","w",newline="",encoding="utf-8") as f:
        w=csv.writer(f)
        w.writerow(["Mode","Database","Indexing_Time_Seconds","Average_Query_Latency_Seconds","Average_Top5_Relevance"])
        for mode,r in [{"Amount included",with_amount}, {"Amount excluded",without_amount}]:
            w.writerow([mode,"FAISS",f"{r['faiss_index_time']:.6f}",f"{r['faiss_avg_latency']:.6f}",f"{r['faiss_relevance']:.4f}"])
            w.writerow([mode,"ChromaDB",f"{r['chroma_index_time']:.6f}",f"{r['chroma_avg_latency']:.6f}",f"{r['chroma_relevance']:.4f}"])

```

```

with open(OUTPUT_DIR/"top5_comparison.txt","w",encoding="utf-8") as f:
    for i in range(3):
        f.write(f"QUERY {i+1}: {QUERIES[i]}\n")
        f.write("FAISS:\n")
        for r in with_amount["faiss_results"][i]:
            f.write(f"{r['Claim_ID']} | {r['Incident_Type']} | INR {r['Claim_Amount_INR']}\n")
        f.write("ChromaDB:\n")
        for r in with_amount["chroma_results"][i]:
            f.write(f"{r['Claim_ID']} | {r['Incident_Type']} | INR {r['Claim_Amount_INR']}\n")
        f.write("\n")
        f.write("SPECIAL CASE: "+SPECIAL_QUERY+"\nFAISS:\n")
        for r in with_amount["special_f"]: f.write(f"{r['Claim_ID']} | {r['Incident_Type']} | INR {r['Claim_Amount_INR']}\n")
        f.write("ChromaDB:\n")
        for r in with_amount["special_c"]: f.write(f"{r['Claim_ID']} | {r['Incident_Type']} | INR {r['Claim_Amount_INR']}\n")

def main():
    print("Insurance Claim Precedent Search")
    print("Dataset:", DATASET)
    print("Embedding model:", MODEL_NAME)
    print("Top-k:", TOP_K)

    print("\nRun 1: claim amount included")
    with_amount=benchmark(True)

    print("\nRun 2: claim amount excluded")
    without_amount=benchmark(False)

    save_outputs(with_amount,without_amount)

    print("\nBENCHMARK WITH AMOUNT INCLUDED")
    print(f"Records: {len(with_amount['rows'])}")
    print(f"FAISS indexing: {with_amount['faiss_index_time']:.6f} s")
    print(f"ChromaDB indexing: {with_amount['chroma_index_time']:.6f} s")
    print(f"FAISS average query latency: {with_amount['faiss_avg_latency']:.6f} s")
    print(f"ChromaDB average query latency: {with_amount['chroma_avg_latency']:.6f} s")
    print(f"FAISS average Top-5 relevance: {with_amount['faiss_relevance']:.4f}")
    print(f"ChromaDB average Top-5 relevance: {with_amount['chroma_relevance']:.4f}")

    print("\nSPECIAL CASE:", SPECIAL_QUERY)
    print("FAISS TOP-5")
    for r in with_amount["special_f"]: print(r["Claim_ID"],r["Incident_Type"],r["Claim_Amount_INR"])
    print("CHROMADB TOP-5")
    for r in with_amount["special_c"]: print(r["Claim_ID"],r["Incident_Type"],r["Claim_Amount_INR"])

    print("\nAMOUNT DISTORTION CHECK")
    print(f"FAISS relevance, amount included: {with_amount['faiss_relevance']:.4f}")
    print(f"FAISS relevance, amount excluded: {without_amount['faiss_relevance']:.4f}")
    print(f"ChromaDB relevance, amount included: {with_amount['chroma_relevance']:.4f}")
    print(f"ChromaDB relevance, amount excluded: {without_amount['chroma_relevance']:.4f}")
    print("\nOutputs saved in:",OUTPUT_DIR)

if __name__=="__main__":
    main()

```

Dataset records: 240

Embedding model: sentence-transformers/all-MiniLM-L6-v2
Top-k: 5

AMOUNT INCLUDED

FAISS indexing time: 0.000187 seconds
ChromaDB indexing time: 0.127724 seconds
FAISS average query latency: 0.000074 seconds
ChromaDB average query latency: 0.003163 seconds
FAISS average Top-5 relevance: 1.0000
ChromaDB average Top-5 relevance: 1.0000

SPECIAL CASE

Query: rear-end collision causing rear bumper and trunk damage

FAISS TOP-5

CLM0024	Rear-end collision	INR 798607
CLM0017	Rear-end collision	INR 140735
CLM0005	Rear-end collision	INR 524110
CLM0008	Rear-end collision	INR 798947
CLM0021	Rear-end collision	INR 39802

CHROMADB TOP-5

CLM0024	Rear-end collision	INR 798607
CLM0017	Rear-end collision	INR 140735
CLM0005	Rear-end collision	INR 524110
CLM0008	Rear-end collision	INR 798947
CLM0021	Rear-end collision	INR 39802

AMOUNT DISTORTION CHECK

FAISS relevance with amount: 1.0000
FAISS relevance without amount: 1.0000

10. Ten Semantic Search Queries

1. rear-end collision with damage to the rear bumper and trunk
2. vehicle hit from the side at an intersection with door damage
3. windshield cracked by road debris
4. car damaged by heavy rain and flood water
5. vehicle stolen from a parking area
6. engine compartment fire caused extensive vehicle damage
7. hailstorm caused dents on the roof and bonnet
8. parked car vandalised with broken windows and scratches
9. motorcycle collision causing front-end damage
10. vehicle hit while parked in a parking area

12. Top-5 Comparison – Three Queries

Use outputs/top5_comparison.txt and compare the returned claim IDs. Record overlaps and differences.

Query	FAISS Top-5	ChromaDB Top-5	Overlap	Reason for Difference
Query 1	Paste actual results	Paste actual results	Calculate	Explain using similarity/ranking
Query 2	Paste actual results	Paste actual results	Calculate	Explain using similarity/ranking
Query 3	Paste actual results	Paste actual results	Calculate	Explain using similarity/ranking

13. Special Case – Same Incident, Different Monetary Amounts

Special query: rear-end collision causing rear bumper and trunk damage.

The dataset contains rear-end collision claims with low, medium and very high monetary amounts. The program first embeds text containing the claim amount and then repeats the experiment with the amount removed. The two runs can be compared to identify whether the amount changes retrieval relevance or ranking.

14. Result Discussion

Complete this section after execution. State the measured indexing times, average query latencies, relevance values, top-5 overlaps, and what changed in the special case when claim amounts were removed.

15. Recommendation

Use the measured benchmark results to select the database. FAISS is suitable when fast local vector search and simple indexing are priorities. ChromaDB is useful when persistent collections and database-style vector management are more important. The final recommendation should be based on the actual measured results.

16. Result

FAISS and ChromaDB are benchmarked using the same sentence embeddings and the same ten semantic queries. The special-case experiment evaluates whether very different claim amounts distort similarity.

17. Conclusion

The experiment provides a practical comparison of FAISS and ChromaDB for insurance claim precedent retrieval. The benchmark and special-case results are used to determine which database better fits the selected use case.